LangChain 기본 구성요소 (실습)

3-1. 왜 LangChain인가
3-2. Model, Prompt, OutputParser (실습)
3-3. 구조화 출력 (실습)

3-1. 왜 LangChain인가

> 직접 API 호출 코드와 나란히 비교해 LangChain의 표준화를 이해해봅니다.

LangChain은 LLM 기반 애플리케이션을 위한 오케스트레이션 프레임워크입니다.

SDK로 직접 호출도 가능합니다. 하지만 기능이 늘수록 직접 짜야 할 코드가 늘어납니다. 모델 교체, 메모리, 구조화 출력, 스트리밍을 모델 제공사(provider)마다 따로 처리해야 하기 때문입니다.

LangChain은 이 공통 작업을 Runnable이라는 하나의 표준 인터페이스로 묶었습니다. Runnable은 모든 부품을 같은 방식으로 실행하게 맞춘 규격입니다.

따라하기

1. 같은 작업을 boto3 직접 호출과 LangChain으로 나란히 실행해 비교합니다. 아래 코드를 `day1_practice/why_langchain.py` 로 저장하고 실행합니다.

```
# why_langchain.py - 직접 호출 vs LangChain 비교
import boto3
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import SystemMessage, HumanMessage

load_dotenv()
MODEL = "us.anthropic.claude-sonnet-4-5-20250929-v1:0"
text = "LangChain is the future of AI development"

# --- A. SDK 직접 호출: provider 전용 형식 ---
client = boto3.client("bedrock-runtime", region_name="us-east-1")
resp = client.converse(
    modelId=MODEL,
    system=[{"text": "너는 짧고 간결한 번역가야."}],
    messages=[{"role": "user", "content": [{"text": f"{text}를 한국어로 번역
해."}]}],
)
print("[SDK 직접]", resp["output"]["message"]["content"][0]["text"])

# --- B. LangChain: 표준 인터페이스 ---
messages = [
    SystemMessage(content="너는 짧고 간결한 번역가야."),
    HumanMessage(content=f"{text}를 한국어로 번역해."),
]
llm = ChatBedrockConverse(model=MODEL, region_name="us-east-1")
print("[LangChain]", llm.invoke(messages).content)

# 모델을 다른 provider로 바꿔도 messages와 invoke 코드는 그대로입니다
# from langchain_openai import ChatOpenAI
# llm = ChatOpenAI(model="gpt-5.4-mini")  # 동일하게 동작
```

결과는 같지만 B는 모델 객체만 갈아 끼우면 다른 provider에서도 동작합니다. 모든 채팅 모델이 `BaseChatModel` 이라는 같은 표준 인터페이스를 따르기 때문입니다.

막힐 때

- LangChain이 오히려 복잡해 보입니다.
  단건 호출만 보면 그렇습니다. 체인을 엮고 메모리를 붙이고 도구를 끼우는 시점부터 이점이 보이는데 4, 5일차에 해당 내용은 직접 확인하실 수 있습니다.

확인: 두 방식의 번역 결과가 같으면 성공입니다.

3-2. Model, Prompt, OutputParser (실습)

> 체인의 세 부품을 각각 따로 실행해 입출력 타입을 확인해봅니다.

LangChain 체인의 3대 기본 부품입니다.

1. Model: 실제 모델 호출 객체입니다. 메시지를 받고 `AIMessage`를 돌려줍니다.
2. PromptTemplate: 변수 자리를 비워 둔 프롬프트 틀입니다.
   템플릿 하나와 입력 리스트로 100건이든 1000건이든 같은 로직으로 처리하는 것이 LLM을 함수처럼 쓰는 핵심입니다.
3. OutputParser: 모델의 `AIMessage` 응답을 코드에서 쓰기 좋은 형태로 바꿉니다.
   가장 기본인 `StrOutputParser`는 `.content`만 뽑아 문자열로 만듭니다.

따라하기

1. 세 부품을 각각 실행해 출력 타입을 확인합니다.
   아래 코드를 `day1_practice/components.py`로 저장하고 실행합니다.

```
# components.py - 세 부품을 각각 실행해 입출력 타입 확인
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# --- 부품 1: PromptTemplate ---
# 입력: dict, 출력: 완성된 프롬프트 (PromptValue)
prompt = PromptTemplate.from_template("{text}를 {language}로 번역해줘.")
filled = prompt.invoke({"text": "Good morning", "language": "한국어"})
print("1) prompt 출력 타입:", type(filled).__name__)
print("   내용:", filled.to_string())

# --- 부품 2: Model ---
# 입력: PromptValue(또는 메시지, 문자열), 출력: AIMessage
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
ai_message = llm.invoke(filled)
print("2) llm 출력 타입:", type(ai_message).__name__)
print("   내용:", ai_message.content)

# --- 부품 3: OutputParser ---
# 입력: AIMessage, 출력: str
parser = StrOutputParser()
text = parser.invoke(ai_message)
print("3) parser 출력 타입:", type(text).__name__)
print("   내용:", text)

# --- 대화형 템플릿: ChatPromptTemplate ---
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "너는 삼성SDS {team} 담당자 톤으로 대답하는 어시스턴트야."),
    ("human", "{question}"),
])
msgs = chat_prompt.invoke({"team": "IT지원팀", "question": "VPN 설정 방법 알려줘."})
print("4) chat_prompt 출력:", msgs.to_messages())
```

먼저 화면에 찍힌 네 줄이 각각 무엇인지부터 봅니다.

화면에 찍힌 것	설명	비유
`1) StringPromptValue`	값이 채워진 프롬프트	모델에 보낼 **질문지 완성본**
`2) AIMessage`	모델의 응답 객체	답변과 토큰 사용량이 든 **상자**
`3) str`	텍스트만 꺼낸 것	상자를 열어 꺼낸 **알맹이**
`4) [SystemMessage, HumanMessage]`	두 개의 메세지 리스트	대화 형태 질문지 (**아직 안 보냄**)

1) PromptTemplate → StringPromptValue

`{text}를 {language}로 번역해줘.` 라는 틀에 dict 값을 넣어 `Good morning를 한국어로 번역해줘.` 를 만들었습니다. 그냥 문자열이면 될 것을 왜 `StringPromptValue` 라는 별도 타입으로 감쌌을까요?

이 값을 어느 모델에 줄지 아직 모르기 때문입니다. 채팅 모델에는 메시지 리스트로 줘야 하고 예전의 텍스트 완성형 모델(구세대 모델 등)에는 문자열로 줘야 합니다.
PromptValue는 그 둘 어느 쪽으로도 변환할 수 있는 중간 형태입니다. `to_string()` 은 그중 문자열 쪽으로 꺼내 본 것일 뿐입니다.

Q. 2번과 3번 내용이 똑같이 나왔는데 맞나요?

네, 정상입니다. 파서는 답을 바꾸는 게 아니라 **상자를 열어 알맹이만 꺼내는** 역할입니다. 내용은 같고 타입만 달라집니다.

Q. 1번은 왜 그냥 문자열이 아닌가요?

이 질문지를 어느 모델에 줄지 아직 안 정해졌기 때문입니다. 채팅 모델은 메시지 리스트를 받고 예전 모델은 문자열을 받는데 `StringPromptValue` 는 둘 다로 바꿀 수 있는 중간 형태입니다. `to_string()` 은 그중 문자열로 꺼내 본 것입니다.

Q. 4번은 왜 답변이 없나요?

모델을 부르지 않았기 때문입니다. 질문지를 만드는 데까지만 하고 멈춘 단계입니다. 출력의 `additional_kwargs={}` 처럼 빈 항목들도 아직 모델을 거치지 않아 채워질 게 없어서입니다.

다시 한번 도식화로 정리하면 아래 흐름입니다.

```
dict   →   PromptValue   →   AIMessage   →   문자열
     프롬프트              모델              파서
```

앞 부품의 출력 타입이 뒤 부품의 입력 타입과 정확히 맞물립니다.

막힐 때

- `KeyError: 'language'` 에러 발생
  템플릿에 선언한 변수와 `invoke` 에 넘긴 dict의 키가 일치하지 않을 때 납니다.
- 프롬프트 안에 진짜 중괄호를 쓰고 싶습니다.
  JSON 예시를 템플릿에 넣을 때는 `{{` 와 `}}` 로 이스케이프합니다.
- `.invoke()` 결과에 `.content` 가 없다는 오류 발생.
  이미 파서를 거친 문자열에 `.content` 를 또 붙인 경우입니다. 지금 다루는 값의 타입이 무엇인지 확인하는 습관이 중요합니다.

(참고) 버전에 따른 차이

3번 파서의 출력 타입이 `str` 이 아니라 `TextAccessor` 로 찍힐 수도 있습니다.

최신 langchain-core가 문자열을 한 번 감싼 형태인데 `str` 을 상속받은 클래스라 `isinstance(x, str)` 가 True이고 `.upper()` 나 슬라이싱, f-string 삽입까지 문자열과 똑같이 동작합니다.
그냥 문자열로 생각하고 쓰면 됩니다. Java로 치면 `String` 을 기대하는 자리에 `String` 을 상속한 하위 타입이 온 것과 같습니다.

확인: 네 단계의 출력 타입이 StringPromptValue, AIMessage, str(또는 TextAccessor), 메시지 리스트로 출력되면 성공입니다.

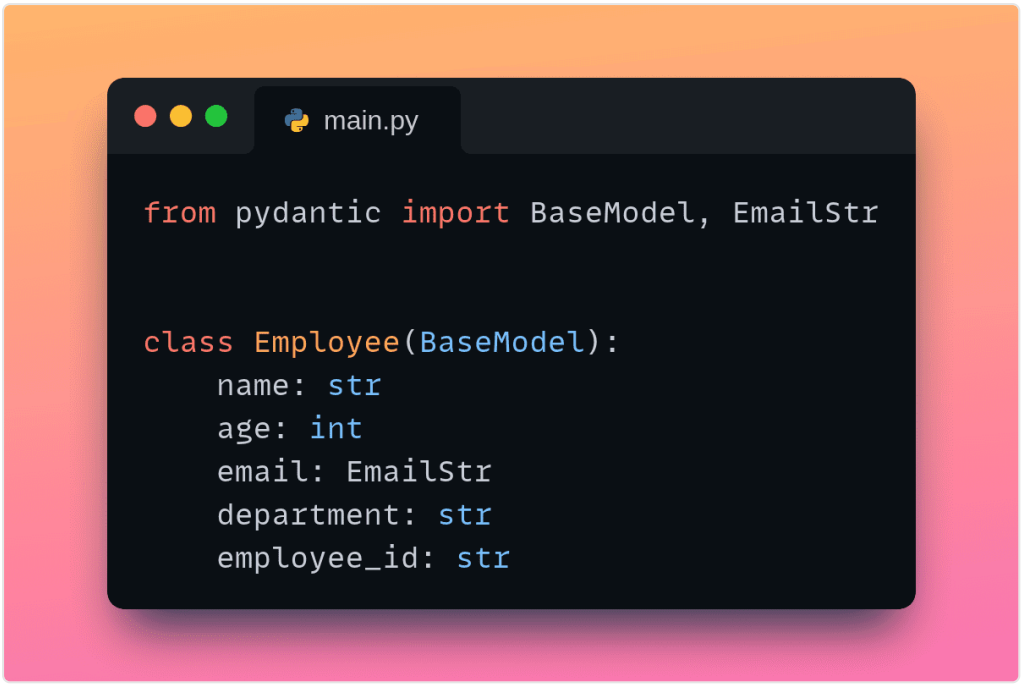
> 부품들의 입출력 타입이 서로 맞물려 체인이 된다는 원리를 익혔습니다.

3-3. 구조화 출력 (실습)

> 모델 응답을 JSON 스키마로 받아 코드로 바로 처리할 수 있게 만들어봅니다.

모델의 답을 코드로 후속 처리하려면 출력 형식이 일정해야 합니다. 프롬프트에 JSON으로 답하라고 적는 것만으로는 불안정합니다. (확률에 기댈수는 없죠!)

LangChain의 해법은 Pydantic 스키마 + JsonOutputParser 조합입니다. Pydantic은 파이썬의 데이터 검증 라이브러리로 Java로 치면 DTO 클래스에 필드 검증이 붙은 것과 비슷하다고 할 수 있습니다.


main.py
from pydantic import BaseModel, EmailStr
class Employee(BaseModel):
name: str
age: int
email: EmailStr
department: str
employee_id: str

따라하기

1. 삼성SDS 사내 문의 텍스트를 분류용 JSON으로 구조화합니다. 아래 코드를 `day1_practice/structured_output.py` 로 저장하고 실행해봅닌다.

```
# structured_output.py - 응답을 JSON 스키마로 받기
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from langchain_aws import ChatBedrockConverse
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import JsonOutputParser

load_dotenv()

# 1) 출력 스키마 정의 - Field의 description이 모델에게 전달되는 설명입니다
class InquiryAnalysis(BaseModel):
    category: str = Field(description="문의 분류: 'IT지원', '인사', '총무' 중 하
나")
    urgency: str = Field(description="긴급도: '높음', '보통', '낮음' 중 하나")
    summary: str = Field(description="문의 내용 한 줄 요약")

# 2) 파서 생성 - 스키마 기반 형식 지침을 자동 생성합니다
parser = JsonOutputParser(pydantic_object=InquiryAnalysis)

# 3) 프롬프트에 형식 지침 끼워 넣기
#    partial_variables: 사용자 입력이 아닌 값을 미리 채워 두는 자리입니다
prompt = PromptTemplate(
    template="다음 사내 문의를 분석해줘.\n\n문의: {inquiry}\n\n{format_instruction
s}",
    input_variables=["inquiry"],
    partial_variables={"format_instructions": parser.get_format_instructions
()},
)

llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)
chain = prompt | llm | parser

result = chain.invoke({"inquiry": "노트북이 갑자기 안 켜지는데 오후에 고객 발표가 있습니
다. 빨리 도와주세요."})

print("타입:", type(result).__name__)
print("전체:", result)
print("분류만:", result["category"])
```

결과가 진짜 dict이므로 곧장 조건 분기, DB 저장, 후속 API 호출에 쓸 수 있습니다.

막힐 때

- 가끔 파싱 오류(`OutputParserException`)가 납니다.
  모델이 형식을 어긴 경우입니다. temperature를 0으로 고정하여 처리할 수도 있고
  `OutputFixingParser` 로 감싸 파싱 실패 시 LLM이 형식을 한 번 더 고치게 할 수도 있습니다.
- `format_instructions` 를 `input_variables` 에 넣었더니 KeyError 발생.
  형식 지침은 사용자 입력이 아니므로 `partial_variables` 에 넣습니다.

확인: 결과 타입이 dict이고 `result["category"]` 로 값을 꺼낼 수 있으면 성공입니다.

> 스키마와 파서로 출력 형식을 강제하는 방법을 익혔습니다.

▼ LangChain은 왜 태어났고 왜 욕을 먹었나

2022년 10월, Harrison Chase가 주말에 혼자 만들어 올린 게 시작이었습니다. 한 달 뒤 ChatGPT가 터지면서 사용자가 한꺼번에 몰렸습니다.

처음엔 욕을 좀 먹었습니다. "API 한 줄이면 되는 걸 왜 감싸냐"는 거였는데, 단건 호출만 놓고 보면 맞는 말이었습니다.

코드가 커지면 이야기가 달라집니다.

- 모델을 갈아 끼워도 위쪽 코드는 그대로 둡니다.
- invoke, stream, batch를 부품마다 다시 짜지 않아도 됩니다.
- 조립한 체인이 다시 부품이 되어 더 큰 구조로 들어갑니다.

2023년에 LCEL이 나오면서 성격이 아예 바뀌었습니다. 감싸는 라이브러리에서 조립 규격으로요. 4번째 세션에서 쓰는 파이프 연산자가 그 결과물입니다.

이후 흐름 제어는 LangGraph로 떨어져 나갔고, Agent 만드는 함수는 langchain.agents의 create_agent로 정리됐습니다. 각각 3일차와 5일차에 직접 만납니다.

쓸지 말지는 결국 한 지점에서 갈립니다. 단건 호출로 끝나면 SDK가 낫고, 부품을 갈아 끼울 일이 생기면 프레임워크가 이깁니다.